

Требования к программам

1. В программе должны быть реализованы следующие структуры данных:

- Контейнер данных объектов типа student:

```
#include <stdio.h>
#include <string.h>
#include <memory>

enum class io_status
{
    success,
    eof,
    format,
    memory,
    open,
    create,
};

class student
{
private:
    std::unique_ptr<char []> name;
    int value = 0;
public:
    student () = default;
    student (const student& x) = delete;
    // По умолчанию: переместить все поля класса - подходит
    student (student&& x) = default;
    ~student () = default;
    student& operator= (const student& x) = delete;
    // По умолчанию: присвоить с перемещением все поля класса - подходит
    student& operator= (student&& x) = default;
    void print (FILE * fp = stdout) const
    {
        fprintf (fp, "%s %d\n", name.get (), value);
    }
    io_status read (FILE * fp = stdin)
    {
        const int LEN = 1234;
        char n[LEN];
        int v;

        if (fscanf (fp, "%s%d", n, &v) != 2)
            return io_status::format;
        erase ();
        return init (n, v);
    }
    int cmp (const student& x) const
    {
        // Используем переопределенную функцию сравнения с nullptr
        if (name == nullptr)
        {
            if (x.name != nullptr)
```

```

        return -1;
        return value - x.value;
    }
    if (x.name.get () == nullptr)
        return 1;
    int res = strcmp (name.get (), x.name.get ());
    if (res)
        return res;
    return value - x.value;
}
int operator< (const student& x) const { return cmp (x) < 0; }
int operator<= (const student& x) const { return cmp (x) <= 0; }
int operator> (const student& x) const { return cmp (x) > 0; }
int operator>= (const student& x) const { return cmp (x) >= 0; }
int operator== (const student& x) const { return cmp (x) == 0; }
int operator!= (const student& x) const { return cmp (x) != 0; }

private:
    io_status init (const char * n, int v)
    {
        value = v;
        if (n != nullptr)
        {
            size_t len = strlen (n);
            name = std::make_unique<char []> (len + 1);
            if (name != nullptr)
            {
                for (size_t i = 0; i <= len; i++)
                    name[i] = n[i];
            }
            else
                return io_status::memory;
        }
        return io_status::success;
    }
    void erase ()
    {
        value = 0;
        name.reset ();
    }
};

```

- Красно-черное дерево (RB-дерево)

```

template <class T> class rb_tree;
template <class T>
class rb_tree_node : public T
{
    enum colors
    {
        invalid,
        red,
        black,
    };
};

```

```

private:
    rb_tree_node * left    = nullptr;
    rb_tree_node * right   = nullptr;
    rb_tree_node * parent = nullptr;
    colors color = invalid;
public:
    rb_tree_node () = default;
    ...
    friend class rb_tree<T>;
};
template <class T>
class rb_tree
{
private:
    rb_tree_node<T> * root = nullptr;
public:
    rb_tree () = default;
    ...
    io_status read (FILE * fp = stdin)
        { ... }
    void print (int r, FILE *fp = stdout) const
        { ... }
};

```

2. Достаточно реализовать только функцию добавления элементов в дерево с сохранением его структуры. Эта функция вызывается при чтении элементов. Дерево удаляется в деструкторе целиком, поэтому функцию удаления элемента из дерева с сохранением его структуры можно не реализовывать.
3. Все функции в задании являются членами класса "дерево".
4. Программа должна получать все параметры в качестве аргументов командной строки. Аргументы командной строки:
 - 1) r – максимальное количество выводимых уровней в дереве,
 - 2) k – параметр задачи,
 - 3) `filename` – имя файла, откуда надо прочитать дерево.

Например, запуск

```
./a.out 4 3 a.txt
```

означает, что RB-дерево надо прочитать из файла `a.txt`, выводить не более 4-х уровней дерева, и вычислить результат задач для $k = 3$.

5. Класс "дерево" должен содержать функцию ввода дерева из указанного файла.
6. Ввод дерева из файла. В указанном файле находится дерево в формате:

```

Слово-1  Целое-число-1
Слово-2  Целое-число-2
...
Слово-n  Целое-число-n

```

где слово – последовательность алфавитно-цифровых символов без пробелов. Длина слова неизвестна, память под него выделяется динамически. Все записи в файле различны (т.е. нет двух, у которых совпадают все поля). RB-дерево заполняется как упорядоченное дерево. **Никакие другие функции, кроме функции ввода дерева, не используют упорядоченность дерева.** Концом ввода считается конец файла. Программа должна выводить сообщение об ошибке, если указанный файл не может быть прочитан или содержит данные неверного формата.

7. Класс "дерево" должен содержать подпрограмму вывода на экран не более чем r уровней дерева. Эта подпрограмма используется для вывода исходного дерева после его инициализации. Подпрограмма выводит на экран не более, чем r уровней дерева, где r – параметр этой подпрограммы (аргумент командной строки). Каждый элемент дерева должен печататься на новой строке и так, чтобы структура дерева была понятна.
8. Поскольку дерево не изменяется функциями из задач, то для всех задач надо сделать **одну функцию** `main`, в которой создается объект `rb_tree<student>`, вводится из указанного файла, выводится указанное число уровней на экран, вызываются все функции задач, выводятся результаты и время их работы; затем удаляется этот объект. После компиляции должен получиться один исполняемый файл `a.out`, а не несколько.
9. Схема функции `main`:

```
int main (int argc, char *argv[])
{
    // Ввод аргументов командной строки
    ...
    rb_tree<student> *a = new rb_tree<student>;
    // Работа с деревом rb_tree<student>
    ...
    delete a;
    return 0;
}
```

10. Вывод результата работы функции в функции `main` должен производиться по формату:

```
printf ("%s : Task = %d K = %d Result = %d Elapsed = %.2f\n",
        argv[0], task, k, res, t);
```

где

- `argv[0]` – первый аргумент командной строки (имя образа программы),
- `task` – номер задачи (1–5),
- `k` – параметр k командной строки,
- `res` – результат работы функции, реализующей решение этой задачи,
- `t` – время работы функции, реализующей решение этой задачи.

Вывод должен производиться в точности в таком формате, чтобы можно было автоматизировать обработку запуска многих тестов.

Задачи

1. Написать функцию – член класса "RB-дерево", получающую в качестве аргумента целое число k , и возвращающую целое значение, равное количеству элементов типа `student` в поддеревьях, имеющих не более k вершин.
2. Написать функцию – член класса "RB-дерево", получающую в качестве аргумента целое число k , и возвращающую целое значение, равное количеству элементов типа `student` в поддеревьях, имеющих не более k уровней.
3. Написать функцию – член класса "RB-дерево", получающую в качестве аргумента целое число k , и возвращающую целое значение, равное количеству элементов типа `student` в поддеревьях, имеющих не более k узлов в любом уровне.
4. Написать функцию – член класса "RB-дерево", получающую в качестве аргумента целое число k , и возвращающую целое значение, равное количеству элементов типа `student` в его k -м уровне.
5. Написать функцию – член класса "RB-дерево", получающую в качестве аргумента целое число k , и возвращающую целое значение, равное количеству элементов типа `student` во всех ветвях длины k , начиная с корня.